

Copy everything below, paste it into a new file, save it as `nexafreight\_plan.html`, then open it in any browser. It is fully self-contained — no internet connection needed to view it. ``html

NexaFreight Control Tower

Complete Final Plan · Architecture, Stack, Data, ML, Build Phases · Budget: \$0 · Solo Developer · August 2026

TABLE OF CONTENTS

- 1. What This Product Is
- 2. Technology Stack
- 3. Folder Structure
- 4. Database Schema
- 5. Data Sources
- 6. Core Concepts
- 7. Adapter Pattern
- 8. Ship Tracking — AISStream.io
- 9. Flight Tracking — Simulated
- 10. Truck Tracking — Road Interpolation
- 11. Sea Routes — searoute-py
- 12. Shipment Consolidation
- 13. Disruption Detection
- 14. Alert Engine
- 15. Rerouting Engine
- 16. Financial Engine
- 17. ML Models
- 18. LLM Copilot
- 19. Background Workers
- 20. API Contract
- 21. Frontend Architecture
- 22. Data Pipeline Scripts
- 23. Phase-by-Phase Build Plan
- 24. Environment Variables
- 25. Makefile
- 26. Failure Handling
- 27. Provenance Rule
- 28. Changes From Original Plan
- 29. One-Page Summary

1. What This Product Is

NexaFreight Control Tower ingests 180,000 DataCo orders, consolidates them into realistic multi-modal shipments, assigns real vessels tracked via AISStream WebSocket, simulated flights on real geodesic arcs, and interpolated trucks on real road geometries. It monitors every shipment in real-time, detects disruptions automatically from port congestion data, AIS deviation, or manual entry, evaluates the financial impact across every order inside the affected shipment, generates 3 rerouting options with full cost/SLA/CO2 analysis, presents them to an operator for approval, logs every decision immutably, and feeds those decisions back as training data.

INDUSTRY CATEGORY

This is a

LOGISTICS CONTROL TOWER (LCT)

. Companies like Flexport, project44, FourKites, SAP TM, and Kinaxis sell enterprise versions for millions of dollars. NexaFreight is a portfolio-grade open-source implementation of the same category.

The Core Workflow

```
ORDER placed by customer
↓
Multiple ORDERS consolidated into one SHIPMENT (physical container)
↓
SHIPMENT assigned a multi-leg ROUTE (first mile → sea/air → last mile)
↓
Real ASSETS assigned (vessel MMSI, flight arc, truck road polyline)
↓
Live EVENTS update shipment progress (AIS, simulation, manual)
↓
DISRUPTION detected (port congestion, vessel delay, strike)
↓
ALERT fired: ETA impact + per-order SLA check + financial exposure
↓
REROUTING ENGINE generates 3 options (accept / divert / modal shift)
↓
FINANCIAL ENGINE scores each option (cost + SLA penalties + CO2)
↓
OPERATOR approves one option
↓
SYSTEM executes: updates legs, increments route_version, writes audit log
↓
Decision becomes training data for next ML cycle
```

The Key Rule

ALWAYS REMEMBER

Physical movement = per

SHIPMENT

(one container).

Financial impact = per

ORDER

(each has its own SLA and penalty).

Rerouting decision = weighs

ALL ORDERS

inside the shipment together.

2. Technology Stack

Backend (Python 3.11)

PACKAGE	VERSION	PURPOSE
<code>fastapi</code>	0.115.0	Web framework
<code>uvicorn[standard]</code>	0.30.6	ASGI server
<code>sqlalchemy</code>	2.0.36	Async ORM
<code>aiosqlite</code>	0.20.0	Async SQLite driver
<code>alembic</code>	1.13.3	Database migrations
<code>duckdb</code>	1.1.2	OLAP analytics queries
<code>pydantic</code>	2.9.2	Data validation
<code>pydantic-settings</code>	2.5.2	Typed env var loading
<code>python-jose[cryptography]</code>	3.3.0	JWT auth
<code>passlib[bcrypt]</code>	1.7.4	Password hashing
<code>websockets</code>	13.1	AISSream WebSocket
<code>apscheduler</code>	3.10.4	Background cron jobs
<code>lightgbm</code>	4.5.0	ML delay + ETA models
<code>shap</code>	0.46.0	ML explainability
<code>statsforecast</code>	1.7.5	Demand forecasting
<code>searoute</code>	1.4.3	Maritime routing (PINNED)

PACKAGE	VERSION	PURPOSE
<code>geopy</code>	2.4.1	Geodesic calculations
<code>shapely</code>	2.0.6	Geometry interpolation
<code>openrouteservice</code>	2.3.3	Road routing API client
<code>google-generativeai</code>	0.8.3	Gemini Flash free LLM
<code>ollama</code>	0.3.3	Local LLM fallback
<code>structlog</code>	24.4.0	Structured JSON logging

Frontend (TypeScript)

PACKAGE	VERSION	PURPOSE
<code>react</code>	18.3.1	UI framework
<code>vite</code>	5.4.8	Build tool
<code>@vis.gl/react-maplibre</code>	8.1.1	React wrapper for MapLibre GL JS
<code>maplibre-gl</code>	6.x	Map rendering (WebGL)
<code>@tanstack/react-query</code>	5.59.0	Server state management
<code>zustand</code>	5.0.0	UI state management
<code>recharts</code>	2.13.0	Charts for analytics
<code>tailwindcss</code>	3.4.13	Styling

Why These Choices

CHOICE	REJECTED ALTERNATIVE	REASON
SQLite +aiosqlite	PostgreSQL	Zero server ops. Same Alembic migrations. One line to upgrade later
DuckDB	Pandas aggregations	Columnar OLAP, reads SQLite + Parquet directly, zero ops
APScheduler	Celery + Redis	No broker needed. Handles cron + interval. Zero ops
StatsForecast	Prophet	100x faster, no PyStan/cmdstan dependency
SSE (Server-Sent Events)	WebSocket for frontend	Simpler, auto-reconnects natively, HTTP/2 compatible
Corridor alternatives	OR-Tools global graph	Realistic, shippable in 1 week, how real 3PLs work
AIStream.io WebSocket	VesselAPI REST	VesselAPI free = 150 calls/month. AIStream = unlimited push
Gemini Flash + Ollama	GPT-4o / Groq	Gemini Flash = 1M tokens/day free. Ollama = fully offline

CHOICE	REJECTED ALTERNATIVE	REASON
MapTiler free tier	Self-hosted tiles	100k loads/month free. Self-hosting needs 100GB + tile server
OpenRouteService cached	OSRM self-hosted	OSRM needs 64GB RAM for global graph. ORS free = 2000 req/day

3. Folder Structure

```
nexafreight/
|
├─ pyproject.toml
├─ .env.example
├─ .gitignore
├─ .pre-commit-config.yaml      ← ruff, mypy, gitleaks
├─ README.md
├─ Makefile
|
├─ alembic/
|   ├── env.py
|   ├── script.py.mako
|   └── versions/
|       └── 001_initial_schema.py
|
├─ data/
|   ├── raw/
|   │   ├── dataco/DataCoSupplyChainDataset.csv
|   │   ├── port_activity/port_activity.csv
|   │   ├── port_performance/maritime_port_performance.csv
|   │   ├── unlocode/unlocode.csv
|   │   └── ais_historical/ais_2024_pacific.parquet
|   └── processed/
|       ├── shipments.parquet
|       ├── legs.parquet
|       ├── sea_routes.parquet
|       └── road_routes.parquet
|
├─ models/
|   ├── delay_classifier/
|   │   ├── model.joblib
|   │   ├── feature_schema.json
|   │   └── metadata.json
|   ├── eta_quantile/
|   │   ├── model.joblib
|   │   ├── feature_schema.json
|   │   └── metadata.json
|   └── demand_forecast/
```

```
|      |─ model.joblib
|      |─ metadata.json
|
|─ scripts/
|   |─ 01_ingest_dataco.py
|   |─ 02_ingest_unlocode.py
|   |─ 03_ingest_port_data.py
|   |─ 04_consolidate_shipments.py
|   |─ 05_plan_routes.py
|   |─ 06_assign_vessels.py
|   |─ 07_assign_flights.py
|   |─ 08_download_ais_history.py
|   |─ 09_train_delay_classifier.py
|   |─ 10_train_eta_model.py
|   |─ 11_train_demand_forecast.py
|
|─ src/
|   |─ nexafreight/
|   |   |─ __init__.py
|   |   |─ config.py
|   |   |─ database.py
|   |   |─ main.py
|   |   |
|   |   |─ models/
|   |   |   |─ base.py
|   |   |   |─ user.py
|   |   |   |─ location.py
|   |   |   |─ order.py
|   |   |   |─ order_item.py
|   |   |   |─ shipment.py
|   |   |   |─ leg.py
|   |   |   |─ vessel.py
|   |   |   |─ position_report.py
|   |   |   |─ event.py
|   |   |   |─ disruption.py
|   |   |   |─ alert.py
|   |   |   |─ reroute_option.py
|   |   |   |─ decision.py
|   |   |   |─ port_daily_stat.py
|   |   |   |─ audit_log.py
|   |   |
|   |   |─ schemas/
|   |   |   |─ auth.py
|   |   |   |─ shipment.py
|   |   |   |─ alert.py
|   |   |   |─ disruption.py
|   |   |   |─ reroute.py
|   |   |   |─ position.py
|   |   |   |─ analytics.py
|   |   |
|   |   |─ adapters/
```

```

├── protocols.py
├── feed/
│   ├── aisstream.py      ← REAL: AISStream WebSocket
│   ├── replay_ais.py     ← REPLAYED: Historical AIS Parquet
│   ├── replay_flight.py  ← SIMULATED: Geodesic arc
│   ├── truck_sim.py      ← SIMULATED: Road polyline
│   └── mock.py           ← MOCK: Tests only
├── routing/
│   ├── sea_route.py
│   ├── road_route.py
│   └── air_route.py
├── llm/
│   ├── gemini.py
│   └── ollama.py
├── services/
│   ├── consolidation.py
│   ├── route_planner.py
│   ├── position_tracker.py
│   ├── disruption_detector.py
│   ├── alert_engine.py
│   ├── reroute_engine.py
│   ├── financial_engine.py
│   ├── sla_checker.py
│   ├── decision_executor.py
│   └── copilot.py
├── ml/
│   ├── registry.py
│   ├── delay_classifier.py
│   ├── eta_model.py
│   └── demand_forecast.py
├── routers/
│   ├── auth.py
│   ├── shipments.py
│   ├── map.py
│   ├── alerts.py
│   ├── disruptions.py
│   ├── decisions.py
│   ├── analytics.py
│   └── copilot.py
├── workers/
│   ├── ais_listener.py
│   ├── port_monitor.py
│   ├── sla_monitor.py
│   └── position_interpolator.py
├── frontend/
└── package.json

```

```
|   ├── vite.config.ts
|   ├── tsconfig.json
|   └── src/
|       ├── main.tsx
|       ├── App.tsx
|       ├── api/
|           ├── shipments.ts
|           ├── alerts.ts
|           ├── analytics.ts
|           └── sse.ts
|       ├── components/
|           ├── map/
|               ├── FreightMap.tsx
|               ├── ShipLayer.tsx
|               ├── FlightLayer.tsx
|               ├── TruckLayer.tsx
|               └── RouteLayer.tsx
|           ├── alerts/
|               ├── AlertPanel.tsx
|               ├── AlertCard.tsx
|               └── RerouteOptions.tsx
|           ├── shipments/
|               ├── ShipmentList.tsx
|               ├── ShipmentDetail.tsx
|               └── OrderTable.tsx
|           └── analytics/
|               ├── FinancialScorecard.tsx
|               ├── DemandForecast.tsx
|               └── ESGReport.tsx
|       ├── pages/
|           ├── Dashboard.tsx
|           ├── ShipmentDetail.tsx
|           ├── AlertCenter.tsx
|           ├── Analytics.tsx
|           └── AuditLog.tsx
|       └── store/
|           └── useAppStore.ts
|
└── tests/
    ├── conftest.py
    ├── unit/
    │   ├── test_consolidation.py
    │   ├── test_route_planner.py
    │   ├── test_financial_engine.py
    │   ├── test_reroute_engine.py
    │   └── test_sla_checker.py
    └── integration/
        ├── test_shipment_api.py
        ├── test_alert_lifecycle.py
        └── test_decision_workflow.py
```


4. Database Schema

Reference Tables

<u>locations</u>	<u>ports</u>	<u>vessels</u>
id (PK)	id (PK)	id (PK)
locode (UNIQUE)	locode (FK→locations)	mmsi (UNIQUE)
name	name	imo
country_code	lat / lon	name
lat / lon	port_type	call_sign
location_type	max_draft_m	vessel_type
PORT AIRPORT	facilities_json	gross_tonnage
WAREHOUSE ICD	avg_dwell_hours	operator
provenance=CALIBRATED	demurrage_free_days	flag_country
source=UNECE_UNLOCODE	demurrage_rate_usd	typical_lanes_json
	congestion_threshold	

Order Domain

<u>orders</u>	<u>order_items</u>
id (PK)	id (PK)
dataco_order_id (UNIQUE)	order_id (FK→orders)
customer_id	product_name
customer_segment	product_category
origin_locode (FK→locations)	quantity
dest_locode (FK→locations)	unit_price
shipping_mode	total_value
order_date	weight_kg
sla_days	cargo_class (GENERAL PERISHABLE HAZMAT)
sla_deadline (computed)	discount_pct
contracted_revenue	profit_per_item
sla_penalty_pct	provenance = DERIVED
shipment_id (FK→shipments)	
provenance = DERIVED	
source = DATACO_CSV	

Shipment Domain

<u>shipments</u>
id (PK)

```

reference_number (UNIQUE)          e.g. "NF-2024-004023"
origin_locode / dest_locode (FK→locations)
primary_mode                      SEA | AIR | ROAD | RAIL
cargo_class                      GENERAL | PERISHABLE | HAZMAT
total_value_usd / total_weight_kg / total_volume_cbm
strictest_sla_deadline            tightest deadline of all orders
planned_departure / planned_arrival
revised_eta                      updated when disruptions hit
actual_arrival
status                          PLANNED|IN_TRANSIT|AT_PORT|DELIVERED|CANCELLED
current_leg_id (FK→legs)
planned_freight_cost_usd / planned_co2_kg
route_version (INT)              increments on every reroute
provenance = DERIVED

legs
——
id (PK)
shipment_id (FK→shipments)
sequence (INT)                  1, 2, 3, 4, 5 in order
route_version (INT)             matches shipment.route_version
mode                            SEA|AIR|ROAD|RAIL|PORT
leg_type                        FIRST_MILE|MAIN_HAUL|ORIGIN_HANDLING|DEST_HANDLING|LAST_MILE
origin_locode / dest_locode (FK→locations)
vessel_id (FK→vessels, nullable)
carrier_name / flight_number
route_geometry (TEXT)           GeoJSON LineString – cached at build time
route_distance_km / route_distance_nm
route_quality                   COMPUTED | APPROXIMATE
planned_departure / planned_arrival
actual_departure / actual_arrival / revised_arrival
planned_cost_usd / planned_co2_kg
demurrage_free_days / demurrage_daily_rate_usd
status                          PLANNED|IN_PROGRESS|COMPLETED|CANCELLED|REPLACED
provenance                      REAL|DERIVED|SIMULATED

INDEX: (shipment_id, route_version, sequence)
INDEX: (vessel_id)
INDEX: (status, mode)

```

Tracking Domain

```

position_reports
——
id (PK)
asset_type                      VESSEL | AIRCRAFT | TRUCK
asset_id                       MMSI | flight_number | leg_id
leg_id (FK→legs, nullable)
lat / lon

```

```

speed_knots / heading_deg / nav_status
reported_at / received_at (TIMESTAMP)
provenance          REAL | REPLAYED | SIMULATED
source              AISSTREAM | OPENSKY_REPLAY | ROAD_INTERPOLATION

```

NOTE: Keep only last 24h in SQLite. Archive to Parquet weekly.
INDEX: (asset_type, asset_id, reported_at DESC)

events

```

_____
id (PK)
shipment_id (FK) / leg_id (FK, nullable)
event_type          SHIPMENT_CREATED|PICKED_UP|ARRIVED_AT_PORT|
                    LOADED_ON_VESSEL|CUSTOMS_CLEARED|POSITION_UPDATE|
                    DISRUPTION_DETECTED|REROUTED|DELIVERED
occurred_at / location_locode / description / source / metadata_json
INDEX: (shipment_id, occurred_at DESC)

```

Alert & Decision Domain

<pre> disruptions _____ id (PK) type PORT_CONGESTION VESSEL_DELAY WEATHER STRIKE CANAL_CLOSURE CUSTOMS_HOLD affected_locode estimated_delay_h delay_confidence started_at / resolved_at source AIS_DEVIATION PORT_DATA MANUAL </pre>	<pre> alerts _____ id (PK) shipment_id (FK) disruption_id (FK) severity LOW MED HIGH CRIT status OPEN ACKNOWLEDGED OPTIONS_GENERATED DECISION_PENDING APPROVED EXECUTED sla_breaches_json financial_exposure_usd created_at acknowledged_by (FK→users) acknowledged_at </pre>	<pre> reroute_options _____ id (PK) alert_id (FK) option_key ACCEPT_DELAY DIVERT MODAL_SHIFT SPLIT name (TEXT) revised_eta additional_cost_usd sla_breaches_count sla_penalty_usd demurrage_usd reroute_cost_usd total_financial_impact co2_delta_kg new_legs_json is_ai_recommended (BOOL) ai_explanation (TEXT) </pre>
<pre> decisions _____ id (PK) alert_id (FK) reroute_option_id (FK) approved_by (FK→users) approved_at </pre>	<pre> audit_log (IMMUTABLE – NEVER UPDATE/DELETE) _____ id (PK) action_type / entity_type / entity_id performed_by (FK→users) performed_at (TIMESTAMP) details_json (TEXT) </pre>	

execution_status
executed_at / notes

5. Data Sources

What You Have — Use These

DATASET	WHAT IT IS	HOW USED
DataCo Smart Supply Chain	180k orders, 30k customers, global lanes	Order backbone, SLA data, revenue/margin, delay labels
Daily Port Activity (IMF)	Daily vessel counts per port	Port congestion detection, disruption trigger
Maritime Port Performance (Kaggle)	Dwell time, berth productivity, turnaround	Calibrate demurrage terms, port delay probability

What You Need To Download — All Free

DATASET	SOURCE	HOW USED
UN/LOCODE	UNECE website	Every location in the system. Origin/dest mapping
World Port Index	NGA / Kaggle mexwell/world-port-index	Port depth, facilities, geographic coverage
NOAA Marine Cadastre AIS	marinecadastre.gov	Historical AIS for ReplayFeedAdapter (US waters)
Danish Maritime Authority AIS	dma.dk (research license)	Alternative historical AIS — global coverage
OpenFlights	openflights.org	Airport pairs + realistic flight durations

DataCo Column Mapping

DATA CO COLUMN	SYSTEM CONCEPT
Shipping Mode "Standard Class"	<code>primary_mode = SEA</code>
Shipping Mode "Second Class"	<code>primary_mode = RAIL</code>
Shipping Mode "First Class"	<code>primary_mode = AIR</code>
Shipping Mode "Same Day"	<code>primary_mode = AIR</code> (express flag)
Days for shipping (real) vs scheduled	Basis for <code>late_delivery_risk</code> ML label
Order Item Total (Sales)	<code>contracted_revenue</code>

DATA CO COLUMN	SYSTEM CONCEPT
Order Profit Per Order	<code>margin</code>
Customer Country / Order Country	Destination / Origin node (map to UN/LOCODE)
Late_delivery_risk	ML classifier target variable
Product Category Name	<code>cargo_class</code> (GENERAL / PERISHABLE / HAZMAT)

6. Core Concepts

The Data Hierarchy

```

CUSTOMER
├── ORDER (one DataCo row = one order)
│   ├── ORDER_ITEMS (products within the order)
│   │   └── SHIPMENT (multiple orders consolidated together)
│   │       ├── LEGS (sequential route segments)
│   │       └── EVENTS (things that happen on each leg)

```

Provenance Labels

Every value in the system must carry one of these labels. No exceptions.

REAL Live feed, confirmed by external source. Example: AIS position from AISStream.

REPLAYED Historical data served as if live. Example: AIS from NOAA Parquet file.

DERIVED Computed from real data. Example: ML prediction, consolidated shipment.

CALIBRATED From authoritative reference data. Example: UN/LOCODE coordinates.

SIMULATED Interpolated or generated. Example: truck position on road polyline.

Status Lifecycles

```

Shipment:  PLANNED → IN_TRANSIT → AT_PORT → IN_TRANSIT → DELIVERED
Alert:     OPEN → ACKNOWLEDGED → OPTIONS_GENERATED → DECISION_PENDING → APPROVED → EXECUTED
Leg:       PLANNED → IN_PROGRESS → COMPLETED
           → REPLACED (if rerouted - never deleted)

```

7. Adapter Pattern

Every external dependency implements a Protocol. Swap the adapter, not the business logic. Tests use MockAdapter. Development uses AISStream + ReplayFlight + TruckSim. Production adds no new code.

```
# adapters/protocols.py

from typing import Protocol
from dataclasses import dataclass
from datetime import datetime
from enum import StrEnum

class AssetType(StrEnum):
    VESSEL = "VESSEL"
    AIRCRAFT = "AIRCRAFT"
    TRUCK = "TRUCK"

class Provenance(StrEnum):
    REAL = "REAL"
    REPLAYED = "REPLAYED"
    SIMULATED = "SIMULATED"
    MOCK = "MOCK"

@dataclass(frozen=True)
class PositionReport:
    asset_type: AssetType
    asset_id: str
    lat: float
    lon: float
    speed_knots: float | None
    heading_deg: float | None
    reported_at: datetime
    provenance: Provenance
    source: str

@dataclass(frozen=True)
class FeedHealth:
    adapter_name: str
    is_healthy: bool
    last_success: datetime | None
    messages_received: int
    provenance: Provenance

class FeedAdapter(Protocol):
    async def start(self) → None: ...
    async def stop(self) → None: ...
    async def get_latest_position(self, asset_id: str) → PositionReport | None: ...
    async def health(self) → FeedHealth: ...
```

8. Ship Tracking — AISStream.io

WHY AISSTREAM

Free WebSocket API. No rate limits in the REST sense. Persistent stream — subscribe once, it pushes continuously. Filter by specific MMSIs. No paid tiers. API key is free on registration (GitHub OAuth).

Critical Caveats

- No direct browser connections. Always proxy through your server.
- Connection drops every ~2 minutes sometimes. Build reconnect with exponential backoff.
- Consume messages fast enough or the service drops them.
- From September 2026: uncompressed connections subject to bandwidth limits. Enable `permessage-deflate` compression.
- No SLA or uptime guarantee. Plan for interruptions. Fallback to REPLAYED.

Connection Architecture

```
AISStream.io WSS endpoint
    ↓
AISStreamAdapter (background asyncio task)
- Connect on startup
- Subscribe: FiltersShipMMSI (your 20-50 active vessel MMSIs)
    FilterMessageTypes: ["PositionReport"]
- On each message: parse → update in-memory cache → write to DB
- On disconnect: exponential backoff (1s, 2s, 4s... max 60s)
- Resubscribe within 3 seconds of reconnect
- Track health: last_success, messages_received, is_healthy
    ↓
In-memory cache (dict[MMSI → PositionReport])
    ↓
SSE endpoint pushes all positions to frontend every 5 seconds
```

MMSI Assignment Strategy

Do NOT subscribe to all ships globally. For each unique (origin_port, dest_port) sea lane in your DataCo data: research 2-3 real vessels that operate that trade lane (Maersk/COSCO/Evergreen), look up their MMSI on MarineTraffic vessel finder (free search, no API needed), insert into vessels table. Subscribe to exactly these 20-50 MMSIs via `FiltersShipMMSI`.

9. Flight Tracking — Simulated

WHY NOT OPENSKY LIVE

Auth changed to OAuth2 in March 2026. Raw ADS-B gives lat/lon/callsign but no flight number, airline, or origin/destination. Non-commercial license only. Coverage gaps outside EU/NA.

What Simulated Means — Not Fake

The geometry IS real. Flights fly great-circle arcs. The duration IS real from actual flight schedule data. The speed IS realistic (~900 km/h cruise). You label it **SIMULATED**. The operator knows it is not live ADS-B. The route drawn on the map is geometrically correct.

How It Works

1. For each DataCo AIR lane (e.g. PVG → LAX):
 - Compute geodesic arc between airport coords (geopy)
 - Store as GeoJSON LineString in legs.route_geometry
 - Get realistic flight duration from OpenFlights data
 - Assign a real cargo flight number (Cathay Cargo, FedEx, DHL)
2. When leg status = IN_PROGRESS:
 - progress = elapsed_time / planned_duration_hours (0.0 → 1.0)
 - position = interpolate_along_arc(origin, dest, progress)
 - speed ≈ 485 knots cruise
 - heading = bearing from current to next point
3. position_interpolator worker runs every 30 seconds
 - writes PositionReport(provenance=SIMULATED, source=GEODESIC_ARC)

10. Truck Tracking — Road Interpolation

Problem 1: Route Geometry

Use **OpenRouteService free tier** (40 req/min, 2000 req/day). Profile: **driving-hgv**. You have ~200 unique city pairs in DataCo. Pre-compute ALL 200 at build time. Cache GeoJSON in the DB. Uses ~200 of 2000 daily free requests. Done in one afternoon. Never call again.

Problem 2: Current Position

Interpolate along the cached road polyline using Shapely. This is identical to how Google Maps shows your GPS dot between actual pings.

```
from shapely.geometry import LineString
import json

def get_truck_position(leg, current_time):
    elapsed = (current_time - leg.actual_departure).total_seconds()
    progress = min(elapsed / leg.planned_duration_seconds, 1.0)

    line = LineString(json.loads(leg.route_geometry)["coordinates"])
    point = line.interpolate(progress, normalized=True)

    return PositionReport(
        asset_type = AssetType.TRUCK,
        asset_id = str(leg.id),
        lat = point.y,
        lon = point.x,
        speed_knots = 60.0 / 1.852,      # ~60 km/h average truck
        heading_deg = compute_heading(line, progress),
        reported_at = current_time,
        provenance = Provenance.SIMULATED,
        source = "ROAD_INTERPOLATION"
    )
```

11. Sea Routes — searoute-py

CRITICAL RULE

Pin version hard: `searoute==1.4.3`. Pre-compute at build time only. Store GeoJSON in DB. Never call at request time. The library is unmaintained — it must never be a runtime dependency.

What It Does

Contains a pre-built maritime network graph (Marnet). Includes passages through Suez Canal, Panama Canal, Malacca Strait, Gibraltar, Dover Strait. Runs Dijkstra on the graph and returns a GeoJSON LineString that follows actual ocean routes, avoiding land.

Build-Time Pattern

```
import searoute as sr, json

try:
```

```

route = sr.searoute(
    [origin_port.lon, origin_port.lat], # [lon, lat] - NOTE the order
    [dest_port.lon, dest_port.lat],
    units="naut"
)
geometry      = json.dumps(route["geometry"])
distance_nm   = route["properties"]["length"]
route_quality = "COMPUTED"

except Exception:
    # Fallback: great-circle line
    geometry      = make_great_circle_geojson(origin_port, dest_port)
    distance_nm   = haversine_nm(origin_port, dest_port)
    route_quality = "APPROXIMATE"

# Store in DB. Done. Never call searoute again at runtime.

```

COORDINATE CONVENTION

searoute-py uses [longitude, latitude] (GeoJSON order). geopy uses (latitude, longitude). Never confuse these — they produce wrong routes silently.

12. Shipment Consolidation

Why Consolidation Is Necessary

DataCo has 180,000 individual order rows. In reality you do not send one container per order. Multiple orders going the same direction by the same mode are consolidated into one physical shipment.

Consolidation Rules

Group orders into the same shipment when ALL of these match:

1. Same origin region (country code prefix of UN/LOCODE)
2. Same destination region
3. Compatible shipping mode (all SEA, or all AIR, etc.)
4. Compatible cargo class (no HAZMAT with GENERAL)
5. Departure windows overlap (orders placed within same 3-day window)
6. Combined weight/volume fits in one container

Expected Output

From 180,000 DataCo orders: approximately **5,000–10,000 shipments**. Each contains 10–40 orders. Of those, roughly **47 are ACTIVE** (IN_TRANSIT) at any given moment. This is realistic for a mid-size freight forwarder operations desk.

THE CRITICAL IMPLICATION

The STRICTEST SLA deadline among all orders in the shipment drives the shipment's urgency. If one order has a 20-day SLA and 19 others have 30-day SLA, the shipment must move as if it has a 20-day SLA.

13. Disruption Detection

Source 1: AIS Deviation (Automatic)

```
Every AIS position update triggers:
planned_progress = elapsed_time / planned_duration
actual_progress  = distance_sailed / total_route_distance

if (planned_progress - actual_progress) > 0.10:
    create_disruption(type=VESSEL_DELAY, source=AIS_DEVIATION)
```

Source 2: Port Congestion (Automatic, Daily at 6am)

```
for each port:
    congestion_index = today_vessel_count / rolling_90_day_average

    if congestion_index > 1.4: # 40% above average
        affected = shipments heading to this port (ON_WATER or PLANNED)
        for shipment in affected:
            create_disruption(type=PORT_CONGESTION, source=PORT_DATA)
```

Source 3: Manual Entry (Operator)

```
POST /api/disruptions
{
  "type": "PORT_STRIKE",
  "affected_locode": "NLRTM",
  "started_at": "2025-03-15T00:00:00Z",
  "estimated_delay_hours": 120,
  "description": "Dock workers strike, port closed 5 days",
}
```

```
"source": "MANUAL "
}
```

Disruption Types

TYPE	COMMON SOURCE	TYPICAL DELAY
PORT_CONGESTION	Port activity data	+2 to +7 days
VESSEL_DELAY	AIS deviation	+1 to +5 days
WEATHER	Manual	+1 to +10 days
STRIKE	Manual	Variable
CANAL_CLOSURE	Manual	+10 to +20 days
CUSTOMS_HOLD	Manual	+2 to +14 days

14. Alert Engine

What The Alert Engine Computes

For each disruption → affected shipment pair:

1. Revised ETA = current ETA + disruption.estimated_delay_hours
2. For each order: is `revised_eta > order.sla_deadline` ?
3. Days late per order = `max(0, (revised_eta - sla_deadline).days)`
4. SLA penalty per order = `revenue × sla_penalty_pct × days_late`
5. Total SLA penalties = sum across all orders
6. Demurrage = `max(0, port_dwell_days - free_days) × daily_rate`
7. Financial exposure = SLA penalties + demurrage

Severity Thresholds

SEVERITY	CONDITION
CRITICAL	Exposure > \$20,000 OR any order > 5 days late
HIGH	Exposure > \$5,000 OR any SLA breach
MEDIUM	Exposure > \$1,000 OR ETA within 2 days of any SLA
LOW	No SLA breach, informational only

15. Rerouting Engine

WHAT REROUTING ACTUALLY MEANS

Rerouting does NOT redirect a ship mid-ocean. It changes the REMAINING legs of the journey after the current leg completes. The vessel finishes what it is doing. You change what happens next.

The Three Option Types

Option A: Accept Delay

Always included as baseline. Keep original route. Accept the delay and penalties. Use when all SLAs still met, or rerouting costs more than demurrage + penalties.

Option B: Divert to Alternative Node

Change the destination port/airport to avoid the disrupted one. Same primary mode. Different transshipment point. Use when nearby port is clear and last-mile cost is acceptable.

Option C: Modal Shift or Split

Switch from sea to air for remaining journey. OR split the shipment: priority orders go air, rest stay on sea. Use when SLA breach is imminent and the penalty exceeds the air premium.

Corridor Alternatives (Stored in DB, Not Computed Live)

Example for Shanghai → Long Beach:

ALT1: Via Oakland

when_to_use: [PORT_CONGESTION, PORT_STRIKE]
time_delta_hours: -48 (2 days faster via less congested port)
cost_delta_factor: 1.08 (8% more expensive)

ALT2: Via Seattle + Rail

when_to_use: [PORT_CONGESTION, WEATHER, CANAL_CLOSURE]
time_delta_hours: -72
cost_delta_factor: 1.05

ALT3: Air Freight (priority cargo only)

when_to_use: [SLA_CRITICAL, HIGH_VALUE_CARGO]
time_delta_hours: -300 (much faster)
cost_delta_factor: 5.5 (air is 5-6x sea cost per kg)

Scoring Function

For each option:

```
revised_eta          = compute from new legs
sla_breaches         = check all orders vs revised_eta
sla_penalties        = sum per-order penalties (see Financial Engine)
demurrage            = (dwell_days - free_days) × daily_rate
freight_delta        = new_freight_cost - original_planned_cost
reroute_cost         = rebooking + transshipment fees
co2_delta            = new_co2 - original_planned_co2
carbon_cost          = co2_delta × carbon_price_per_kg
total_financial_impact = sum of all above
```

Rank: lowest total_financial_impact = best option

Mark: AI recommendation = best-ranked option that also meets SLA

Show: top 3 options only (always include Accept Delay)

The Split Decision

Evaluate split when: some orders breach SLA and others do not.

```
priority_orders = [o for o if revised_eta > o.sla_deadline]
standard_orders = [o for o if revised_eta ≤ o.sla_deadline]
```

```
total_sla_penalties = sum(calculate_penalty(o) for o in priority_orders)
air_cost            = sum(estimate_air_freight(o) for o in priority_orders)
```

```
if total_sla_penalties > air_cost:
    RECOMMEND SPLIT:
        priority_orders → air freight → meets all SLAs
        standard_orders → stay on sea → all SLAs met anyway
        net_savings = total_sla_penalties - air_cost
```

16. Financial Engine

Complete Cost Breakdown Per Option

```
Revenue          = agreed freight rate (from DataCo Sales column)
Freight cost      = carrier rate × distance × weight
Port handling     = origin terminal + destination terminal fees
Demurrage        = max(0, dwell_days - free_days) × daily_rate
SLA penalties     = per-order: revenue × penalty_pct × days_late
Rerouting surcharge = rebooking fee + transshipment cost
Carbon cost       = co2_kg × carbon_price_per_kg (default $0.08/kg)
```

Gross margin = Revenue - sum(all costs)
Decision = option with lowest total_financial_impact
that satisfies SLA constraints

CARBON PRICE DEFAULT

\$0.08 per kg CO2 = ~\$80 per tonne. This approximates the EU ETS shadow price. Configurable via environment variable.

17. ML Models

Model 1: Delay Classifier (LightGBM)

PROPERTY	VALUE
Algorithm	LightGBM Classifier
Target	<code>late_delivery_risk</code> (0/1) from DataCo
Train split	Before 2017-06-01 (TIME-BASED, not random)
Val split	2017-06-01 to 2017-12-31
Test split	2018-01-01 onwards
Tuning	Optuna, 50 trials, maximize val ROC-AUC
Requirement	MUST beat naive baseline (historical late rate per lane)
Explainability	SHAP TreeExplainer — top 5 factors per prediction

Key features: shipping_mode_encoded, origin/dest_country_code, planned_transit_days, cargo_value_usd, cargo_class, origin/dest_port_congestion_index, departure_month, departure_weekday, distance_km, sla_buffer_days.

```
# Output per prediction:
{
  "delay_probability": 0.73,
  "is_high_risk": true,
  "top_factors": [
    {"feature": "dest_port_congestion_index", "impact": 0.31, "direction": "increases_delay"},
    {"feature": "departure_month", "impact": 0.18, "direction": "increases_delay"},
    {"feature": "sla_buffer_days", "impact": -0.14, "direction": "reduces_delay"}
  ],
  "model_version": "abc123de",
```

```
"provenance": "DERIVED"
}
```

Model 2: ETA Quantile Regressor (LightGBM)

Three separate models: **P10** (optimistic), **P50** (median), **P85** (conservative). Target = `actual_transit_days - planned_transit_days` (the residual, not absolute). Evaluated on pinball loss. P10-P85 interval must contain actual result ~75% of the time. **Use P85 for SLA risk checks** — protects operator from surprises.

Model 3: Demand Forecast (StatsForecast AutoETS)

Weekly order volume per (product_category × origin_region × dest_region). 30/60/90-day horizon. Point forecast + confidence intervals. Used for capacity pre-negotiation, not individual shipment decisions. StatsForecast is 100x faster than Prophet with no PyStan dependency.

Model Artifacts (Saved Per Model)

```
models/delay_classifier/
  model.joblib          ← The trained model
  feature_schema.json   ← Feature names + types (inference uses this)
  metadata.json         ← {
                          "git_sha": "abc123def456",
                          "training_data_hash": "sha256: ... ",
                          "trained_at": "2024-11-01T10:00:00Z",
                          "val_roc_auc": 0.847,
                          "test_roc_auc": 0.831,
                          "baseline_roc_auc": 0.621,
                          "n_train_samples": 145000
                        }
```

18. LLM Copilot

PRIORITY WARNING

Build this LAST (Phase 6). The map, ML, alerts, and rerouting are more impressive and more important for any portfolio review or pitch. Do not let the copilot eat your time budget.

What It Is

A structured AI analyst scoped to one shipment at a time. It **explains and recommends**. It CANNOT trigger actions. It CANNOT approve decisions. It CANNOT see other users' shipments.

Stack

- **Primary:** Google Gemini Flash — free, 15 req/min, 1M tokens/day
- **Fallback:** Ollama local (llama3.1:8b, ~8GB RAM, works fully offline)
- **Switch:** `USE_GEMINI=false` in `.env` → automatic fallback

Context Injection Pattern

```
def build_copilot_context(shipment_id: int) → str:
    # Pull structured data from DB
    shipment    = db.get_shipment(shipment_id)
    alerts      = db.get_active_alerts(shipment_id)
    prediction   = ml.predict_delay(shipment_id)
    financials  = finance.get_pnl(shipment_id)
    sop_rules   = rules.get_applicable(shipment)

    return f"""
    You are a logistics analyst for NexaFreight Control Tower.
    Answer using ONLY the data below.
    If you don't know, say "I don't have that information."
    Never invent shipment IDs, vessel names, or financial figures.
    You cannot approve actions – only explain and recommend.

    SHIPMENT STATE:      {json.dumps(shipment.to_dict())}
    ACTIVE ALERTS:       {json.dumps([a.to_dict() for a in alerts])}
    ML PREDICTION:       delay_prob={prediction.delay_probability:.1%}
                        ETA range: {prediction.p10_eta} to {prediction.p85_eta}
    FINANCIAL SNAPSHOT: revenue=${financials.revenue:,.0f}
                        margin={financials.margin_pct:.1%}
                        demurrage_risk=${financials.demurrage_risk:,.0f}
    APPLICABLE RULES:    {json.dumps(sop_rules)}
    """
```

19. Background Workers

All workers run inside the FastAPI process via APScheduler. No Redis. No Celery. No broker. No extra containers.

```
from apscheduler.schedulers.asyncio import AsyncIOScheduler

scheduler = AsyncIOScheduler()
scheduler.add_job(run_port_monitor, 'cron', hour=6, minute=0)
scheduler.add_job(run_sla_monitor, 'interval', minutes=15)
```

```
scheduler.add_job(run_position_interpolator, 'interval', seconds=30)
scheduler.start()
```

WORKER	SCHEDULE	WHAT IT DOES
<code>ais_listener.py</code>	Always running (asyncio task)	Persistent AISStream WebSocket. Reconnects with backoff. Writes to position_reports + cache
<code>port_monitor.py</code>	Daily at 6am	Reads port_daily_stats. Computes congestion_index. Creates DISRUPTION records for affected shipments
<code>sla_monitor.py</code>	Every 15 minutes	Checks all IN_TRANSIT shipments. Creates ALERT if new SLA breach detected
<code>position_interpolator.py</code>	Every 30 seconds	Interpolates truck + flight positions. Writes SIMULATED PositionReports

20. API Contract

AUTH

```
POST /api/auth/login      → {access_token, token_type, role}
POST /api/auth/refresh    → {access_token}
GET  /api/auth/me         → {id, email, role}
```

SHIPMENTS

```
GET /api/shipments      → {items: Shipment[], total, page}
    ?status=IN_TRANSIT &mode=SEA &alert=true &page=1&size=20
GET /api/shipments/{id} → ShipmentDetail (legs, orders, events)
GET /api/shipments/{id}/route → {legs: [{mode, geometry, quality}]}
GET /api/shipments/{id}/predict → DelayPrediction + ETAPrediction
GET /api/shipments/{id}/financials → FinancialSnapshot
GET /api/shipments/{id}/events → Event[]
```

MAP

```
GET /api/map/positions/stream → SSE stream (text/event-stream)
    Pushes POSITION_UPDATE every 5 seconds
GET /api/map/positions/snapshot → PositionReport[] (initial load)
GET /api/map/routes → GeoJSON FeatureCollection (all active routes)
GET /api/map/ports → Port[] with current congestion_index
GET /api/map/feed-health → FeedHealth[] per adapter
```

ALERTS

```
GET /api/alerts      → Alert[]
    ?status=OPEN &severity=HIGH,CRITICAL
GET /api/alerts/{id} → AlertDetail (with reroute_options)
PATCH /api/alerts/{id}/acknowledge → {status: "ACKNOWLEDGED"}
```

DISRUPTIONS

GET	/api/disruptions	→ Disruption[]
POST	/api/disruptions	→ Create manual (OPERATOR role required)
GET	/api/disruptions/{id}	→ DisruptionDetail
DECISIONS		
GET	/api/alerts/{id}/options	→ RerouteOption[] (3 options, full financials)
POST	/api/alerts/{id}/approve	→ Approve one option
	body: {option_id, notes}	→ requires OPERATOR role
	returns: {status, new_eta}	
GET	/api/decisions	→ Decision[] (audit history)
GET	/api/decisions/{id}	→ DecisionDetail
ANALYTICS (DuckDB-powered)		
GET	/api/analytics/scorecard	→ {total_shipments, on_time_pct, margin, demurrage ... }
GET	/api/analytics/demand	→ DemandForecast[] per category × lane
GET	/api/analytics/esg	→ {total_co2_kg, by_mode, trend}
GET	/api/analytics/sla	→ SLA performance by mode, carrier, lane
GET	/api/analytics/financial	→ P&L: revenue, freight, demurrage, penalties
COPILOT		
POST	/api/copilot/ask	→ {answer, model_used, context_used}
	body: {shipment_id, question}	
	auth: required	

21. Frontend Architecture

State Management

STATE TYPE	TOOL	EXAMPLES
Server state	TanStack Query	Shipment lists, alert data, analytics responses
Live positions	Custom SSE hook	Asset positions, updated every 5s via EventSource
UI state	Zustand	Selected shipment, panel open/closed, modal state
Auth state	Zustand	Current user, JWT token (stored in memory, not localStorage)

Pages

PAGE	ROUTE	PRIMARY COMPONENTS
Dashboard	/	FreightMap + ShipmentList + AlertPanel
Shipment Detail	/shipments/:id	ShipmentDetail + OrderTable + CopilotPanel
Alert Center	/alerts	AlertCard list + RerouteOptions modal

PAGE	ROUTE	PRIMARY COMPONENTS
Analytics	/analytics	FinancialScorecard + DemandForecast + ESGReport
Audit Log	/audit	Decision history table

Map Implementation Notes

- MapLibre GL JS v6 ships as ES modules. Configure worker URL before rendering (Vite-specific).
- Use `@vis.gl/react-maplibre` v8.1.1 for React component API.
- Map tiles: MapTiler free tier (dark dataviz style).
- Route lines by mode: blue dashed (sea), orange solid (air), green solid (road).
- Asset markers rotate SVG icon to heading angle.
- Provenance badge:  for SIMULATED,  for REPLAYED. No badge = REAL.

SSE Hook

```
// frontend/src/api/sse.ts
export function useSSEPositions(): Position[] {
  const [positions, setPositions] = useState<Position[]>([]);

  useEffect(() => {
    const es = new EventSource('/api/map/positions/stream');

    es.onmessage = (e) => {
      const data = JSON.parse(e.data);
      if (data.type === 'POSITION_UPDATE') setPositions(data.positions);
    };

    // EventSource auto-reconnects. No manual logic needed.
    return () => es.close();
  }, []);

  return positions;
}
```

22. Data Pipeline Scripts

Run in order. Once. Never again unless source data changes.

```
Script 01: ingest_dataco.py
Input: DataCoSupplyChainDataset.csv
```

Output: orders + order_items tables

- Map shipping_mode → SEA|AIR|RAIL
- Compute sla_deadline = order_date + sla_days
- Map product_category → cargo_class
- Derive sla_penalty_pct by customer_segment
- provenance = DERIVED, source = DATACO_CSV

Script 02: ingest_unlocode.py

Input: unlocode.csv

Output: locations table

- Parse UN/LOCODE 5-char codes
- Convert lat/lon from DDMM format to decimal degrees
- provenance = CALIBRATED, source = UNECE_UNLOCODE

Script 03: ingest_port_data.py

Input: port_activity.csv + maritime_port_performance.csv

Output: ports + port_daily_stats tables

- Fuzzy match port names to UN/LOCODE
- Compute baseline avg_turnaround_hours (90-day average)
- Compute daily congestion_index per port

Script 04: consolidate_shipments.py

Input: orders table

Output: shipments table, orders.shipment_id updated

- Group by (origin_country, dest_country, mode, cargo_class, week)
- Fill containers (max 20 orders or weight limit)
- strictest_sla_deadline = min(order.sla_deadline) per group

Script 05: plan_routes.py

Input: shipments + locations + ports tables

Output: legs table with route_geometry (GeoJSON cached)

- SEA legs: searoute-py → GeoJSON + distance_nm
Failure → great-circle fallback, route_quality=APPROXIMATE
- ROAD legs: OpenRouteService (driving-hgv profile), batch 40/min
- AIR legs: geodesic arc (geopy), duration from OpenFlights
- CO2 per leg (GLEC factors): SEA=6.5, AIR=500, ROAD=62, RAIL=22 g/tonne-km
- Chain planned_departure/arrival through all legs

Script 06: assign_vessels.py

Input: legs (SEA, MAIN_HAUL only)

Output: vessels table + legs.vessel_id updated

- Research real vessels per trade lane (Maersk/COSCO/Evergreen)
- Get MMSI from MarineTraffic free search
- Assign MMSIs round-robin to sea legs

Script 07: assign_flights.py

Input: legs (AIR, MAIN_HAUL only)

Output: legs.flight_number + planned_duration_hours updated

- Pick real cargo flights per airport pair
- Duration from OpenFlights database

Script 08: download_ais_history.py
Input: MMSIs from vessels table
Output: data/raw/ais_historical/*.parquet
- Download from NOAA Marine Cadastre (US waters, free)
OR Danish Maritime Authority (global, research license, free)
- Filter to your MMSIs only
- Convert CSV → Parquet (10x smaller)
- Used by ReplayFeedAdapter when USE_LIVE_AIS=false

Scripts 09–11: Train ML models
- Run after Phase 1 data is complete
- Time-based splits (no data leakage)
- Optuna hyperparameter search
- Must beat naive baseline
- Save model.joblib + feature_schema.json + metadata.json

23. Phase-by-Phase Build Plan

PHASE 0

Foundation

Week 1

Zero data, zero features

Goal: The skeleton everything plugs into

- pyproject.toml with all pinned dependencies
- .env.example with all vars documented
- Full package structure (`src/nexafreight/`, all `__init__.py`)
- config.py — pydantic-settings, all env vars typed
- database.py — async SQLAlchemy 2.0 + aiorsqlite
- All ORM model files (schema defined, empty)
- Alembic init + migration 001_initial_schema.py
- main.py — FastAPI app factory, lifespan, CORS
- All routers as stubs returning 200 {"status": "ok"}
- adapters/protocols.py + adapters/feed/mock.py
- tests/conftest.py + 5 unit tests on financial_engine, all pass
- Makefile: dev, test, migrate, lint, seed, train
- pre-commit hooks: ruff, mypy, gitleaks
- GitHub Actions: on push → lint + test
- Frontend: Vite + React + TypeScript + Tailwind init
- Login page (POST /auth/login, JWT stored in memory)

✅ Done when: `make dev` starts on :8000 · `npm run dev` starts on :5173 · Login works · `make test` passes · Zero secrets in git · `alembic upgrade head` runs clean

PHASE 1

Data Foundation

Week 2–3

Scripts 01–07

Goal: All data ingested, all routes pre-computed

- Run scripts 01 through 07 in order
- Validate: 180k orders → 5k-10k shipments → legs with GeoJSON geometry
- All sea routes: searoute-py (or fallback to great-circle)
- All road routes: OpenRouteService, cached in DB
- All air routes: geodesic arcs computed
- All sea legs: real vessel MMSIs assigned
- Implement: GET /api/shipments, GET /api/shipments/{id}, GET /api/shipments/{id}/route
- Unit tests: test_consolidation.py, test_route_planner.py

✅ Done when: /api/shipments/42/route returns valid GeoJSON · Sea routes go through correct straits · Road routes follow actual roads · Every leg has provenance label

PHASE 2

Live Map

Week 4–5

The WOW moment

Goal: Assets move on a real map

- Script 08: download historical AIS data
- All four feed adapters: AISStream, ReplayAIS, TruckSim, ReplayFlight
- workers/ais_listener.py (persistent asyncio task)
- workers/position_interpolator.py (30s interval)
- SSE endpoint: GET /api/map/positions/stream
- GET /api/map/positions/snapshot + /routes + /ports + /feed-health
- Frontend: FreightMap.tsx (MapLibre GL JS v6 + MapTiler tiles)
- Route lines by mode + asset markers with icons + heading rotation + provenance badges
- Click marker → side panel with shipment summary

✅ Done when: Ships move in real-time · Trucks inch along road polylines · Flights move along curved arcs · No asset crosses land on a sea route · Feed health shows per-adapter status

PHASE 3

ML Models

Week 6–8

Scripts 09–11

Goal: Predict delay + ETA with SHAP explanations

- Train all three models (delay classifier, ETA quantile, demand forecast)
- ml/registry.py loads models at startup from /models/ dir
- ml/delay_classifier.py, ml/eta_model.py, ml/demand_forecast.py wrappers
- GET /api/shipments/{id}/predict → prediction with SHAP factors
- Frontend: risk badges (LOW/MEDIUM/HIGH) + ETA ranges + top SHAP factor on cards
- Analytics page: demand forecast chart by category

✅ Done when: Each model beats naive baseline · SHAP explanations are intuitive · Every prediction has model_version and provenance=DERIVED · Risk badges visible on all active shipments

PHASE 4

Alert + Financial Engine

Week 9–10

Goal: Detect disruptions, compute per-order financial impact

- workers/port_monitor.py + workers/sla_monitor.py
- services/disruption_detector.py + services/alert_engine.py
- services/financial_engine.py + services/sla_checker.py
- POST /api/disruptions (manual entry)
- GET /api/alerts + GET /api/alerts/{id} + PATCH /api/alerts/{id}/acknowledge
- Frontend: AlertPanel.tsx with severity sorting + per-order SLA breakdown table

✅ Done when: Manual disruption → alert within 30 seconds · Alert shows correct financial exposure · Per-order table shows which orders breach · Financial calculations covered by unit tests

PHASE 5

Rerouting Engine

Week 11–12

Goal: 3 options per disruption, operator approves, system executes

- Seed corridor_alternatives table for major DataCo lanes (data not code)
- services/reroute_engine.py (score alternatives, generate 3 options)
- services/decision_executor.py (apply approved decision, update legs)
- GET /api/alerts/{id}/options + POST /api/alerts/{id}/approve
- GET /api/decisions (audit history)

- Frontend: RerouteOptions.tsx (3 cards, ★ AI recommendation, Approve button with confirmation)
- Frontend: AuditLog page

✅ Done when: Inject disruption → 3 options with correct financials · Approve B → legs updated → map shows new route → audit log entry · SPLIT option appears when warranted · Decisions are immutable

PHASE 6

LLM Copilot

Week 13

Goal: Structured AI that explains, never decides

- adapters/llm/gemini.py (Gemini Flash free) + adapters/llm/ollama.py (local fallback)
- services/copilot.py (context builder + LLM caller)
- POST /api/copilot/ask → {answer, model_used, context_used}
- Frontend: collapsible copilot panel in ShipmentDetail

✅ Done when: Answers shipment questions correctly · Declines questions outside its context · Works offline with Ollama (USE_GEMINI=false)

PHASE 7

Analytics + Polish

Week 14–15

Goal: Dashboards, ESG, demo script

- DuckDB integration for all analytics queries (reads SQLite directly)
- GET /api/analytics/scorecard, /demand, /esg, /sla, /financial
- Frontend: FinancialScorecard + DemandForecast + ESGReport charts
- DB indexes on hot query paths
- Feed health dashboard
- 10-minute demo script (pre-staged disruption at minute 3)
- README with architecture diagram + setup instructions
- Rotate all secrets used during development

MINIMUM DEMO-ABLE BUILD

Phases 0 + 1 + 2 + partial 4 = approximately

6 WEEKS

. Delivers: live map with real ships + DataCo shipments + manual disruption trigger + financial alert. Already a demo-worthy product.

24. Environment Variables

```
# .env.example - copy to .env and fill in values

# ——— REQUIRED ———
# python -c "import secrets; print(secrets.token_hex(32))"
SECRET_KEY=your-32-byte-hex-secret-here

# Register free at aisstream.io (GitHub OAuth)
AISSTREAM_API_KEY=your-aisstream-api-key

# Register free at openrouteservice.org
ORS_API_KEY=your-ors-api-key

# Register free at maptiler.com
MAPTILER_API_KEY=your-maptiler-api-key

# ——— DATABASE ———
DATABASE_URL=sqlite+aiosqlite:///./nexafreight.db
DUCKDB_PATH=./nexafreight_analytics.duckdb

# ——— LLM (at least one required for copilot) ———
# Option A: Gemini Flash - free at aistudio.google.com
GEMINI_API_KEY=your-gemini-api-key
USE_GEMINI=true

# Option B: Ollama local (run: ollama pull llama3.1:8b)
# USE_GEMINI=false
# OLLAMA_BASE_URL=http://localhost:11434
# OLLAMA_MODEL=llama3.1:8b

# ——— FEATURE FLAGS ———
DEBUG=true
USE_LIVE_AIS=true          # false = use ReplayFeedAdapter from Parquet

# ——— THRESHOLDS ———
PORT_CONGESTION_THRESHOLD=1.4      # 40% above average triggers alert
VESSEL_DELAY_THRESHOLD_PCT=0.10    # 10% behind schedule triggers alert

# ——— FINANCIAL DEFAULTS ———
CARBON_PRICE_PER_KG_USD=0.08      # ~$80/tonne CO2 (EU ETS shadow price)
DEFAULT_SLA_PENALTY_PCT=0.05      # 5% of order revenue per day late
```

25. Makefile

```
.PHONY: dev dev-frontend test test-cov lint format migrate \
        migrate-new seed train clean install check

dev:
    uvicorn src.nexafreight.main:app --reload --port 8000

dev-frontend:
    cd frontend && npm run dev

test:
    pytest tests/ -v --tb=short

test-cov:
    pytest tests/ --cov=src/nexafreight --cov-report=html

lint:
    ruff check src/ tests/
    mypy src/nexafreight/

format:
    ruff format src/ tests/

migrate:
    alembic upgrade head

migrate-new:
    alembic revision --autogenerate -m "$(name)"

seed:
    python scripts/01_ingest_dataco.py
    python scripts/02_ingest_unlocode.py
    python scripts/03_ingest_port_data.py
    python scripts/04_consolidate_shipments.py
    python scripts/05_plan_routes.py
    python scripts/06_assign_vessels.py
    python scripts/07_assign_flights.py

train:
    python scripts/09_train_delay_classifier.py
    python scripts/10_train_eta_model.py
    python scripts/11_train_demand_forecast.py

clean:
    find . -type d -name __pycache__ -exec rm -rf {} +
    find . -name "*.pyc" -delete
    rm -f nexafreight.db nexafreight_analytics.duckdb
```

```
install:
  pip install -e ".[dev]"
  cd frontend && npm install

check:
  make lint
  make test
  @echo "🟢 All checks passed"
```

26. Failure Handling

FAILURE	DETECTION	RECOVERY
AISStream drops connection	<code>last_success > 5 min</code>	Exponential backoff reconnect. Switch to REPLAYED. Badge turns grey
AISStream bandwidth limit (Sep 2026)	Messages dropped	Enable <code>permessage-deflate</code> compression on WebSocket
ORS rate limit during seed	HTTP 429	Exponential backoff + retry queue. Log failed lanes
searoute-py fails for a port	Exception in script 05	Fallback to great-circle. Set <code>route_quality=APPROXIMATE</code>
ML model not found at startup	FileNotFoundError in registry	Log error. Disable prediction endpoints. Serve 503 with message
DuckDB analytics query slow	Response > 3s	Add DuckDB index on hot columns
SQLite write contention	Concurrent writes	WAL mode enabled in <code>connect_args</code>
Gemini API rate limit	HTTP 429	Automatic fallback to Ollama (adapter pattern)
Ollama not running	Connection refused	503 from copilot endpoint with clear message
AIS position for unknown MMSI	Not in vessels table	Discard silently. Log once with structlog
ORS returns no route	No route found error	Fallback: haversine distance × 1.35 road factor

27. Provenance Rule

THE ONE RULE THAT GOVERNS EVERYTHING

Every value in the system has one of five labels. If you cannot label it, you do not ship it.

LABEL	MEANING	FRONTEND
REAL	Live feed, confirmed by external source. AIS position from AISStream	No badge (clean, trusted)
REPLAYED	Historical data served as if live. AIS from NOAA Parquet file	Grey "R" badge
DERIVED	Computed from real data. ML prediction, consolidated shipment	Labeled as "prediction" — no badge needed
CALIBRATED	From authoritative reference. UN/LOCODE coordinates, port dwell times	No badge (reference data is trusted)
SIMULATED	Interpolated or generated. Truck on road polyline, flight on arc	Yellow "S" badge

Every API response that contains a position, prediction, cost, or ETA must include a **provenance** field. This is enforced by Pydantic schema. If provenance is missing from any response: it is a bug, not a feature.

28. Changes From Original Plan

Removed (10 items)

#	REMOVED	WHY
1	Six Sigma / DPMO / SPC charts	Wrong product category. Manufacturing ops, not freight
2	Docker / Docker Compose	Zero ops needed. SQLite runs without containers
3	Celery + Redis	APScheduler handles background jobs without a broker
4	PostgreSQL	SQLite + Alembic is identical ORM API. One line to upgrade later
5	Self-hosted MapLibre tiles	100GB storage + tile server. Not needed until real traffic
6	Self-hosted OSRM	64GB RAM for global graph extraction. Not viable at \$0
7	VesselAPI REST polling	Free tier = 150 calls/month. Need 67,000+/day
8	OR-Tools Dijkstra on live global graph	Weeks of data engineering. No edge weight data source
9	Facebook Prophet	Heavy C++ deps (PyStan). Replaced by StatsForecast (100x faster)
10	Customer segmentation screen	Not core to control tower pitch

Replaced (10 items)

#	ORIGINAL	REPLACEMENT	REASON
11	VesselAPI REST	AISStream.io WebSocket	Free, real-time, no rate limits, MMSI filter

#	ORIGINAL	REPLACEMENT	REASON
12	OpenSky live API	Simulated flights on geodesic arcs	Auth changed, non-commercial, no flight metadata
13	OSRM self-hosted	OpenRouteService free tier, pre-computed once	ORS free = 2000 req/day, 200 lanes done once
14	Prophet	StatsForecast AutoETS	100x faster, no PyStan dependency
15	OR-Tools global graph	Pre-built corridor alternatives + scoring	Realistic, 1 week to build, how real 3PLs work
16	GPT-4o / Groq	Gemini Flash + Ollama	1M tokens/day free, works fully offline
17	PostgreSQL + Alembic	SQLite + Alembic + DuckDB	Same migrations, zero ops, DuckDB for analytics
18	searoute-py at runtime	searoute-py at build time, cached GeoJSON	Unmaintained — never a runtime dependency
19	Self-hosted tiles Phase 7	MapTiler free tier from Phase 2	100k loads/month, no infra needed
20	OSRM truck simulation	ORS cached polyline + Shapely interpolation	Same result, ORS does the routing, Shapely does the math

Added (20 items)

#	ADDED	WHY
21	AISStream.io WebSocket adapter	Free real-time ship tracking with MMSI filter
22	FeedAdapter Protocol pattern	Swappable adapters. Tests use Mock. Prod uses AISStream
23	Provenance labeling system	Every value knows what it is. No dashboard lies
24	ReplayFeedAdapter	Demo without live API, offline use, scripted scenarios
25	DuckDB for analytics	Columnar OLAP. Reads SQLite + Parquet. Zero ops
26	Feed health endpoint	Operator sees adapter status, staleness, message count
27	Shipment consolidation logic	Orders don't ship individually in real logistics
28	Shipment SPLIT decision	Priority orders air, rest sea, when financially justified
29	Corridor alternatives table	Pre-built rerouting options as data, not code
30	Per-order financial impact	Each order has its own SLA deadline and penalty clause
31	SHAP explanations	ML predictions explain themselves in plain English
32	Quantile ETA model (P10/P50/P85)	Three models, evaluated on pinball loss
33	Immutable audit log	Every decision, actor, timestamp. Never UPDATE or DELETE
34	route_version on shipments + legs	Full rerouting history preserved, queryable

#	ADDED	WHY
35	SSE for map updates	Frontend connects once, backend pushes every 5 seconds
36	APScheduler background workers	Port monitor, SLA monitor, position interpolator
37	Makefile	make dev, test, seed, train, check — developer experience
38	Model metadata.json	git SHA, data hash, metrics, trained_at per model artifact
39	Time-based train/val/test split	Future data cannot leak into training. Simulates production
40	Baseline model requirement	ML model must beat naive predictor before it ships

29. One-Page Summary

What It Is

A Logistics Control Tower. Real product category. Scaled-down open-source version of what Flexport, project44, and FourKites sell for millions. Every feature solves a real operational problem.

Complete Stack

BACKEND
Python 3.11 · FastAPI 0.115 · SQLAlchemy 2.0 async · SQLite + DuckDB · Alembic

ML
LightGBM (delay + ETA) · StatsForecast (demand) · SHAP (explanations) · Optuna (tuning)

ROUTING
searoute-py (sea, build-time) · OpenRouteService (road, cached) · geopy (air geodesic arc)

TRACKING
AISStream WebSocket REAL · Road interpolation SIMULATED · Geodesic arc SIMULATED

LLM
Gemini Flash free (1M tokens/day) · Ollama local fallback (offline demo)

FRONTEND
React 18 · Vite · MapLibre GL JS v6 · @vis.gl/react-maplibre · TanStack Query · Zustand · Recharts · Tailwind

INFRA
Zero Docker. SQLite file. Start with: uvicorn + npm run dev. Total cost: \$0

MAP TILES
MapTiler free tier (100k loads/month) · Dark dataviz style

Build Timeline

PHASE	WEEKS	DELIVERS
0: Foundation	1	Skeleton, auth, CI, zero data

PHASE	WEEKS	DELIVERS
1: Data	2–3	All DataCo ingested, all routes pre-computed in DB
2: Live Map	4–5	Real ships on map (AISStream), simulated trucks + flights
3: ML	6–8	Delay classifier, ETA quantiles, demand forecast
4: Alerts	9–10	Disruption detection, per-order financial impact
5: Rerouting	11–12	3 options, approve, execute, immutable audit log
6: Copilot	13	Scoped LLM, explains not decides
7: Analytics	14–15	DuckDB dashboards, ESG, demand, P&L scorecard

Total Cost: \$0

SERVICE	FREE LIMIT
AISStream.io	Free WebSocket, no call limits, no paid tiers
OpenRouteService	2000 req/day — used only at build time (200 routes, done once)
MapTiler	100,000 tile loads/month
Google Gemini Flash	1,000,000 tokens/day
Ollama	Free, local, no API key required
SQLite + DuckDB	Free, no server, no ops
GitHub Actions	Free for public repos / 2000 min/month private
All Python packages	Open source, MIT or BSD licensed

The Five Most Important Decisions

- 1. **AISStream.io WebSocket** — the only way to get real-time ship tracking at \$0
- 2. **SQLite + DuckDB** — eliminates all ops overhead with zero capability loss
- 3. **searoute-py at build time only** — unmaintained library never touches production
- 4. **Corridor alternatives as data** — realistic, shippable, how real 3PLs actually work
- 5. **Provenance on everything** — operators trust what they see, bugs are immediately obvious

THE ONE RULE

Every value in the system has one of five labels: REAL | DERIVED | CALIBRATED | SIMULATED | REPLAYED.
If you cannot label it, you do not ship it.

...